TRAIL Scout — AI & Data Strategy

Deliverable 10 | AI/data pipeline, architecture, model strategy, privacy and ethics

TRAIL • MEMBER 4 • AI, DATA & FINANCE

Member 4 — AI, Data & Finance Specialist

Project: TRAIL — Cartographic Pedestrian Exploration & Discovery Platform

Author: Member 4 — AI, Data & Finance

Date: October 2026

AI System Name: TRAIL Scout

Primary Model: Google Gemini 1.5 Pro / Flash

Fallback: Local Curated Knowledge Engine (Zero-dependency, 100% offline)

Section 1: Executive Summary

TRAIL Scout is the AI intelligence layer of the TRAIL platform. It transforms a pedestrian explorer's natural-language intent — *"find me quiet coffee spots near Fontainhas"* or *"plan a 2-hour heritage walk with a bookstore"* — into a structured, atmospheric, multi-stop trail dossier.

The AI solves the **core business problem directly**: traditional navigation apps route you to a destination; TRAIL Scout synthesises a living, curated *journey* between experiences.

Section 2: The 8-Stage AI Data Pipeline

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■                  TRAIL SCOUT — AI DATA PIPELINE                  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

  STAGE 1: DATA SOURCE
  ■■■ OpenStreetMap Overpass API (geo-coordinates, place types)
  ■■■ Goa Tourism Heritage Survey 2024 (cultural context)
  ■■■ TRAIL Field Curation (curator clues, vibe tags, opening hours)
  ■■■ Community Submissions (foot-traffic assessments)
        ■
        ▼
  STAGE 2: DATA COLLECTION
  ■■■ OSM Overpass QL queries by bounding box (Panaji lat/lng range)
  ■■■ Manual field reconnaissance (Member 4 + team)
  ■■■ Cross-referencing Google Maps Place API for hours verification
  ■■■ Community form submissions (anonymised, no PII collected)
        ■
        ▼
  STAGE 3: DATA CLEANING
  ■■■ Normalising categories → 12 SpotCategory enum values
  ■■■ Removing duplicate/closed venues (verified October 2026)
  ■■■ Standardising coordinates to 4 decimal places
  ■■■ Filling missing opening hours with "Hours vary — verify on arrival"
  ■■■ Privacy audit: zero PII, zero real customer records in dataset
        ■
        ▼
  STAGE 4: DATA STORAGE
  ■■■ PRIMARY: TypeScript module — `backend/src/data/spots.ts`
```

```
■   ■■■ 21+ Spot objects in curated_spots[] array
■   ■■■ Rich metadata: lat/lng, vibes[], bestTime[], footTraffic
■   ■■■ Utility functions: getSpotsByCategory(), searchSpots()
■■■ MIRROR: `frontend/data/spots.ts` (client-side fallback)
■■■ FORMAT: Strongly-typed TypeScript (Spot interface in types/chat.ts)
      ■
      ▼
STAGE 5: AI / LLM
■■■ PRIMARY: Google Gemini 1.5 Pro (rich reasoning + local nuance)
■■■ SECONDARY: Google Gemini 1.5 Flash (speed-optimised)
■■■ TERTIARY: Google Gemini 2.0 Flash (next-gen, if configured)
■■■ OFFLINE FALLBACK: Deterministic keyword + vibe scoring engine
      ■
      ▼
STAGE 6: USER QUERY
■■■ TrailChatbot.tsx: Free-text conversational interface
■   ■■■ POST /api/chat  { message, history[], locus, maxWalkMinutes }
■■■ AI Discovery Page: Calibration form interface
    ■■■ POST /api/generate-trail  { prompt, locus, timeHours, tags[] }
      ■
      ▼
STAGE 7: AI PROCESSING
■■■ extractConversationContext() — parses intent, time, categories
■■■ getRelevantSpotsWithContext() — scores & retrieves top 5-8 POIs
■■■ buildSystemPromptWithContext() — assembles LLM context window
■■■ callGeminiAPI() — sends to Gemini with conversation history
■■■ generateIntelligentFallback() — offline backup engine
■■■ parseSpotCards() + generateContextualFollowUps() — post-processing
      ■
      ▼
STAGE 8: OUTPUT
■■■ CHAT MODE: ChatResponse { message, spotCards[], suggestedFollowUps[] }
■   ■■■ Rendered as: conversational text + rich SpotCard components
■■■ TRAIL MODE: RichTrail JSON { title, stops[], detailedStops[], metadata }
    ■■■ Rendered as: AISynthesizedDossier (full trail dossier UI)
```

Section 3: Detailed Explanation of Each Stage

Stage 1 — Data Source

Source	Type	Content	Coverage
OpenStreetMap Overpass API	Public API	Geo-coordinates, place type tags, address	Panaji city boundary
Goa Tourism Heritage Survey 2024	Government Report	Cultural significance ratings, heritage site context	Fontainhas Latin Quarter
TRAIL Field Curation	Original Research	Curator notes, vibe tags, opening hours verification	21+ spots across 4 quarters
Community Submissions	Anonymised Form	Foot traffic assessments, hidden gem tips	User-contributed

Privacy Compliance: All data sources are public, anonymised, or synthetically generated. Zero real individual PII is used anywhere in the system.

Stage 2 — Data Collection

Field scouting was conducted across four Panaji neighbourhoods: **Fontainhas**, **Altinho**, **Campal**, and **Miramar**. Each spot was:

- Physically verified to exist and be operating (October 2026).
- Cross-referenced against OpenStreetMap and Google Maps for coordinate accuracy.
- Photographed and noted for opening hours, foot traffic density, and unique characteristics.
- Assigned a `source` tag: `'Field Curation + OSM'`, `'OSM + Community'`, or `'Community Submission'`.

Stage 3 — Data Cleaning & Preprocessing

The raw data went through the following cleaning steps before entering the dataset:

Category normalisation: All places mapped to one of 12 SpotCategory values (`coffee`, `bookstore`, `art`, `street-art`, `nature`, `viewpoint`, `heritage`, `food`, `hidden-gem`, `artisan`, `music`, `culture`).

Coordinate precision: Lat/Lng rounded to 4 decimal places (≈11m precision, sufficient for pedestrian navigation).

Duplicate removal: 4 duplicate or permanently closed venues were excluded during field verification.

Opening hours standardisation: All times in `HH:MM - HH:MM` format; seasonal closures noted.

Privacy audit: No real customer names, phone numbers, email addresses, or any PII included in any field.

Stage 4 — Data Storage

Primary Storage: `backend/src/data/spots.ts`

- Format: TypeScript module exporting `curated_spots: Spot[]`
- Type Definition: `Spot` interface in `backend/src/types/chat.ts`
- 21+ spots with 20+ typed fields each

Data Schema (Spot Interface):

```
interface Spot {
  id: string;                   // Unique identifier (e.g., 'spot-cafe-bodega')
  name: string;                 // Full display name
  shortName?: string;           // Abbreviated name for map pins
  category: SpotCategory;       // One of 12 categories
  categoryTags: string[];       // UI badge tags (e.g., ['COFFEE', 'COURTYARD'])
  description: string;          // 2-sentence atmospheric description
  curatorNote: string;          // Insider tip from field curator
  location: {
    lat: number; lng: number;   // GPS coordinates
    address: string;            // Street address
    quarter: string;            // Neighbourhood (e.g., 'Fontainhas')
    city: string;               // City name
  };
  vibes: string[];              // Mood tags (e.g., ['quiet', 'colonial'])
  footTraffic: FootTraffic;     // 'quiet' | 'moderate' | 'busy'
  bestTime: BestTime[];         // When to visit
  openingHours?: string;        // Operating hours
  estimatedMinutes: number;     // Recommended dwell time
  walkTimeFromCenterMin: number;// Minutes walking from city centre
  distanceFromCenterKm: number; // Distance in km
  indoorOutdoor: string;        // 'indoor' | 'outdoor' | 'both'
  noiseLevel: string;           // 'silent' | 'quiet' | 'moderate' | 'lively'
  highlighted?: boolean;        // Featured spot flag
  source: string;               // Data provenance
}
```

Utility functions available:

```
getSpotsByCategory(category: SpotCategory): Spot[]
getHighlightedSpots(): Spot[]
getSpotsWithinRadius(maxWalkMinutes: number): Spot[]
searchSpots(query: string): Spot[]
```

Stage 5 — AI Model Selection

TRAIL Scout uses a **multi-tier model waterfall** to ensure 100% uptime regardless of API availability:

```
Tier 1: Gemini 1.5 Pro  ■■■■ API key present + high reasoning needed
Tier 2: Gemini 1.5 Flash ■■■ API key present + speed-optimised request
Tier 3: Gemini 2.0 Flash ■■■ API key present + next-gen mode (optional)
Tier 4: Local Fallback ■■■■■■ No API key / API failure (always available)
```

Why Gemini?

- Native multimodal understanding for travel context.
- Excellent multilingual support (English + Konkani/Hindi for Goa context).
- Token efficiency: A 5-spot trail synthesis consumes ~1,200 input + ~450 output tokens ≈ **■0.025 per query**.

- Prompt caching reduces repeated system-prompt costs by ~50%.

System Prompt Strategy:

- Version: `v1.2` (`buildSystemPromptWithContext()`)
- TRAIL Scout is assigned the persona of a *"local Goan urban curator, not a navigation app."*
- Spot context is injected dynamically based on user intent.
- Negative keyword detection prevents recommending excluded spots.

Stage 6 — User Query Interfaces

TRAIL Scout exposes two interaction modes:

Mode A — Conversational Chat (`/api/chat`):

```
User types: "I want quiet coffee with a book for 2 hours this morning"
      ↓
System extracts: { categories: ['coffee', 'bookstore'], time: 'morning', duration: 120, mood: 'quiet' }
      ↓
Top 5 matching spots retrieved from curated_spots[]
      ↓
Context window assembled → Gemini generates curator-voiced response
      ↓
Spot cards + follow-up suggestions returned
```

Mode B — Trail Synthesis (`/api/generate-trail`):

```
User fills calibration form: { prompt, locus, timeHours, tags[], maxDistanceKm }
      ↓
Structured JSON trail dossier generated by Gemini
      ↓
AISynthesizedDossier rendered with full stop-by-stop UI
```

Stage 7 — AI Processing Architecture

Core processing functions in `backend/src/services/aiService.ts`:

Function	Role	Input → Output
`extractConversationContext()`	Intent parsing & memory	`history[], message` → `ConversationContext`
`getRelevantSpotsWithContext()`	Semantic spot retrieval	`context` → `Spot[]` (top 5-8)
`scoreSpotRelevance()`	Relevance scoring engine	`spot, query` → `number` (relevance score)
`buildSystemPromptWithContext()`	LLM prompt assembly	`context, spots[]` → `string` (system prompt)
`callGeminiAPI()`	LLM API call	`prompt, message, history, apiKey` → `{text, modelUsed}`
`generateIntelligentFallback()`	Offline fallback engine	`context, spots[]` → `string` (structured response)
`parseSpotCards()`	Response post-processor	`aiText, spots[]` → `SpotCard[]`
`generateContextualFollowUps()`	UX follow-up generator	`context, spots[]` → `string[]` (3 follow-up prompts)
`cleanChatResponse()`	Response sanitiser	`rawText` → `cleanedText`

Stage 8 — Output Format

Chat Mode Output (`ChatResponse`):

```
{
  "message": "For quiet morning coffee in Fontainhas, I'd start at Cafe Bodega...",
```

```
  "spotCards": [
    {
      "spot": { "id": "spot-cafe-bodega", "name": "Cafe Bodega", ... },
      "matchReason": "Quiet colonial courtyard café, morning hours, artisanal pour-over",
      "relevanceScore": 9.5
    }
  ],
  "suggestedFollowUps": [
    "Is Cafe Bodega walkable from the Latin Quarter?",
    "Are there bookshops nearby I can visit after?",
    "What's the best time to avoid crowds at Cafe Bodega?"
  ],
  "source": "gemini",
  "modelUsed": "gemini-1.5-pro",
  "processingMs": 1240
}
```


Trail Mode Output (`RichTrail`):


```
{
  "id": "ai-trail-fontainhas-morning",
  "title": "Quiet Mornings Through the Latin Quarter",
  "distanceKm": 2.8,
  "durationMinutes": 120,
  "stops": [ { "name": "Cafe Bodega", "estimatedMinutes": 35 }, ... ],
  "detailedStops": [ { "bearing": "15°29'54\"N", "surfaceType": "BASALT PAVING", ... } ]
}
```


Section 4: AI Architecture Summary Card

Property	Value
System Name	TRAIL Scout
Version	1.0.0
Primary Model	Google Gemini 1.5 Pro
Secondary Model	Google Gemini 1.5 Flash
Tertiary Model	Google Gemini 2.0 Flash
Offline Fallback	Deterministic keyword + vibe scoring engine
System Prompt Version	v1.2
Dataset Size	21+ curated POIs across 4 Panaji quarters
API Endpoint (Chat)	`POST /api/chat`
API Endpoint (Trail)	`POST /api/generate-trail`
Avg. API Cost per Query	~■0.025 (~$0.0003)
Target Response Latency	< 2,000ms (Gemini) / < 50ms (Fallback)
Uptime Guarantee	100% (fallback always available)
PII in Dataset	None — fully anonymised and synthetic-safe

Section 5: Data Ethics & Privacy Compliance

TRAIL strictly complies with all competition Data & Privacy requirements.

Requirement	TRAIL Compliance Status
No real bank account numbers	■ Not applicable — no financial PII in any dataset
No Aadhaar / PAN numbers	■ Zero identity documents in any file
No passwords	■ No user credentials stored or used
No private medical records	■ Not applicable

Requirement	TRAIL Compliance Status
No real people's private information	■ All spot data is about public commercial venues
Use anonymised or synthetic data	■ All data is venue-level, not individual-level
Public datasets used	■ OpenStreetMap (ODbL license), Goa Tourism public reports

Data Minimisation Principle: The TRAIL Scout system collects only the minimum data required for trail synthesis. No user queries are stored. No conversation history is persisted server-side. Each API call is stateless.